

Trabalho da Disciplina Laboratório de Programação I

FEN06 04049 – 2025-2

Prof. João Araujo

1 O Desafio: Solucionador do Puzzle Klotski em C

O objetivo deste trabalho é desenvolver um programa em **linguagem C** capaz de resolver o puzzle Klotski (também conhecido como Huarong Dao). O programa deverá ler a configuração inicial do puzzle a partir de um arquivo de texto e, em seguida, fornecer a sequência de movimentos necessários para alcançar a solução desejada.

1.1 Especificação do Arquivo de Entrada

O programa deve ser robusto e capaz de processar um arquivo de entrada de texto com as seguintes características.

Tabela 1: Especificações do Arquivo de Entrada do Puzzle Klotski

Característica	Descrição
Ignorados	Linhas em branco e linhas que começam com # (comentários) devem ser ignoradas.
Peças	Cada peça do puzzle é representada por uma letra do alfabeto.
Peça Alvo	A peça que deve ser deslocada para a posição final é a única representada por uma letra maiúscula .
Outras Peças	As demais peças são representadas por letras minúsculas.
Espaços Vazios	Representados por um ponto (.
Matriz Inicial	O puzzle completo é representado por uma matriz de letras, onde cada letra indica uma peça distinta ou parte de uma mesma peça.
Tamanho Máximo	O programa deve estar preparado para aceitar matrizes de até 10 x 10 .
Solução Almejada	Após a matriz inicial, a próxima matriz representa a configuração final. As posições irrelevantes (que podem conter qualquer peça) são preenchidas com o número 0 (zero).
Grupos de Peças	Um número indica a quantidade de grupos de peças com o mesmo formato. Em seguida, para cada grupo, deve haver uma linha listando as letras que representam essa forma, separadas por espaço. Formatos que possuem apenas uma peça não entram nesta lista.

1.2 Estratégia de Solução

Uma solução viável para este problema é a **Busca em Espaço de Estados** (*State-Space Search*), utilizando uma abordagem de **força bruta** com otimização.

Recomendação Técnica: A busca pode ser computacionalmente intensiva. Para evitar a exploração de caminhos repetidos, é **obrigatório** que o programa armazene

e verifique as configurações de tabuleiro já visitadas. Uma forma de otimizar isso é usar uma estrutura de dados eficiente (como uma árvore de busca ou um *hash set*) para mapear cada configuração única do tabuleiro, permitindo a poda de estados repetidos.

1.3 Sintaxe de Execução

O programa deve ser chamado via linha de comando, respeitando rigorosamente a seguinte sintaxe:

```
nomeProg <nome do puzzle.txt>
```

Assim, se o programa se chama *klotski*, o comando para resolver o quebra-cabeça do *engarrafamento* seria:

```
./klotski engarrafamento.txt
```

2 Exemplos de Entrada

A seguir, dois exemplos de arquivos de entrada para teste.

```
engarrafamento.txt
# engarrafamento
XX.gga
XX.gbh
iicdhh
jjkemn
jkkfmn
# solução
000000
000000
000000
0000XX
0000XX
# grupos
4
a b c d e f
g j
h k
m n
```

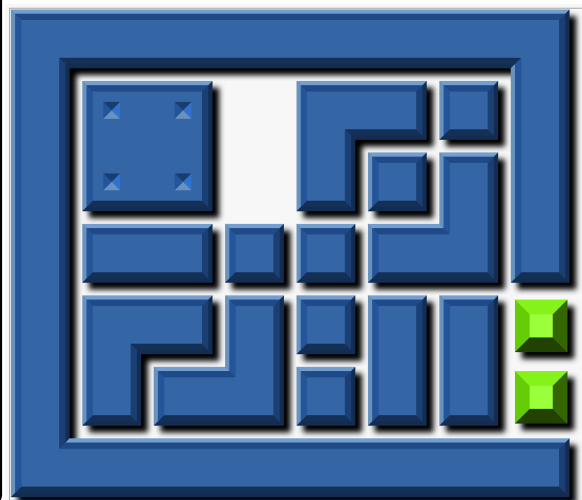


Figura 1: Exemplo de configuração inicial do Klotski.

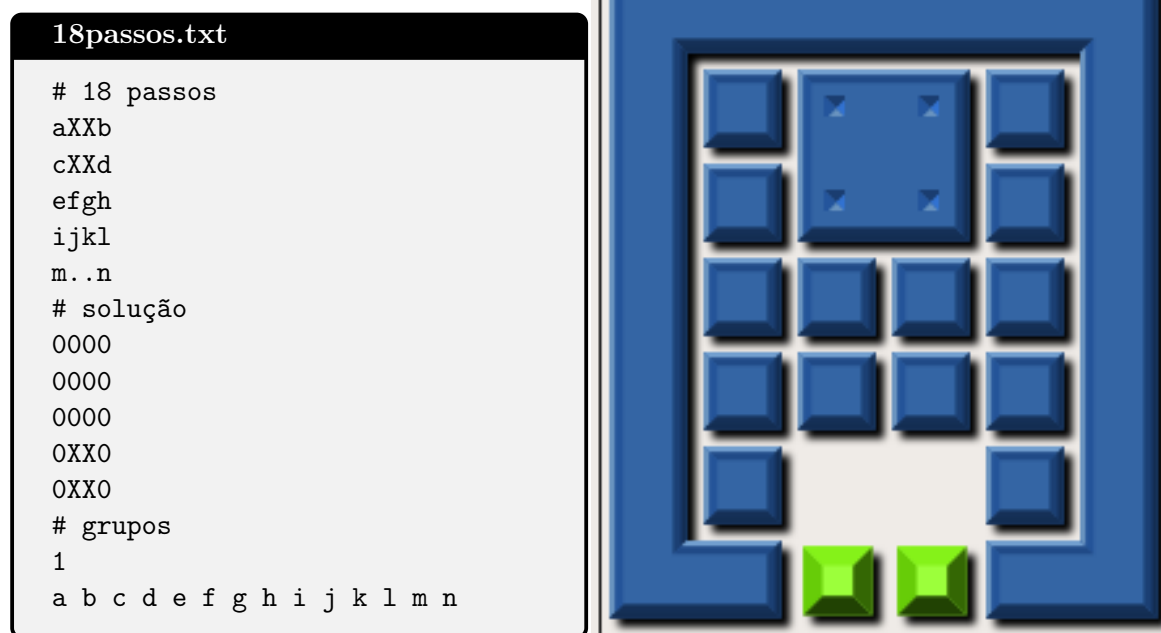


Figura 2: Exemplo de configuração mais simples.

3 Política de Uso de Inteligência Artificial (IA)

Reconhecendo a evolução das ferramentas de desenvolvimento, o uso de Grandes Modelos de Linguagem (LLMs) como ChatGPT, Claude, GitHub Copilot, etc., para o desenvolvimento do solucionador do puzzle Klotski **está permitido**.

No entanto, a permissão vem acompanhada de uma mudança fundamental no critério de avaliação. O objetivo do trabalho deixa de ser apenas “**entregar um código que funciona**” e passa a ser “**demonstrar domínio completo sobre a solução implementada**”.

3.1 O Princípio da Autoria e Responsabilidade

Você pode usar a IA como uma “programadora júnior” ou “parceira de par”, mas **você é o engenheiro responsável**.

- **Erros de lógica**, *memory leaks* (vazamento de memória) ou falhas de segmentação gerados pela IA serão descontados integralmente da sua nota.
- A justificativa “*foi a IA que gerou assim*” **não será aceita** sob nenhuma hipótese.

4 O Novo Formato de Avaliação (100%)

A nota do trabalho será composta por três etapas interdependentes, focadas em avaliar sua autoria e domínio técnico.

4.1 Etapa A: O Código e a Solução (40% da nota)

O programa deve resolver o puzzle respeitando rigorosamente todas as especificações do arquivo de entrada.

Ponto de Atenção: IAs costumam falhar no *parsing* de arquivos com formatação específica. Verifique se o seu código trata corretamente as peças irrelevantes (0) e a peça alvo (XX maiúsculo).

4.2 Etapa B: O Relatório de “Code Review” (20% da nota)

Junto com o código, você deve entregar um documento (PDF) contendo uma análise crítica e técnica do código gerado (ou auxiliado) pela IA. O relatório deve incluir:

1. **Estratégia de *Prompting*:** Um resumo da estratégia usada para instruir a IA, incluindo os *prompts* mais importantes.
2. **Crítica de Erros e Correções:** Identifique pelo menos **dois trechos** onde a IA gerou código errado, ineficiente ou com problemas de memória (*bugs*), e explique **como e por que** você realizou a correção.
3. **Análise da Estrutura de Dados:** Explique como a IA sugeriu armazenar a árvore de estados (ex: lista, *hash map*, vetor) e justifique tecnicamente por que essa escolha é **adequada ou inadequada** para eliminar caminhos repetidos e garantir a eficiência da busca.

4.3 Etapa C: A Validação Presencial / “Live Coding” (40% da nota)

No dia da entrega (ou data a marcar), haverá uma sessão presencial em laboratório para validação do seu domínio sobre o código.

- **O Desafio:** Você receberá uma tarefa de modificação do seu próprio código para ser realizada em **50 a 100 minutos, sem acesso à internet**.
- Exemplos de tarefas possíveis:
 - Alterar a heurística de busca (ex: de BFS para A*).
 - Modificar a saída para desenhar o tabuleiro graficamente a cada passo.
 - Inserir contadores de nós visitados na memória.
 - Adaptar o código para lidar com uma nova regra de movimentação (ex: peças só podem se mover para cima/baixo).

Regra de Ouro: Se o código funciona, mas você não consegue modificá-lo presencialmente para atender a uma demanda simples, assume-se que você não domina a autoria do trabalho. A nota da Etapa C será **zero** e comprometerá a aprovação na disciplina.

5 Recomendações Técnicas Finais (Foco em C)

Dado que o trabalho exige o uso da linguagem C, preste atenção especial aos seguintes pontos, que são frequentemente negligenciados por LLMs:

1. **Gerenciamento de Memória (malloc/free):** IAs esquecem frequentemente de liberar memória em estruturas de árvore complexas. Use **ferramentas como Valgrind** para garantir que não há vazamentos (*memory leaks*) e inclua essa verificação no seu processo de desenvolvimento.

2. **Argumentos de Linha de Comando:** Garanta que a IA respeitou a sintaxe de execução `./klotski <arquivo>`. O uso de nomes de arquivos fixos (*hardcoding*) não será aceito.
3. **Complexidade e Legibilidade:** O enunciado sugere força bruta com poda de repetidos. Se a IA gerar um código excessivamente complexo que você não consegue explicar, **peça para simplificar**. Código “mágico” que o aluno não entende é a maior causa de reprovação na etapa presencial. Priorize a **legibilidade** e a **simplicidade** da sua solução.

Desejo a todos um excelente trabalho!